Digital Bank

Project map

Every file: what it does, why it exists, and what it connects to.

As of 2026-10-14 · commit db27196

How to read this

Each folder has a section, and each file has a card. A card answers three questions: **what it does**, **why we need it**, and **what it connects to**. The **IMPORTS** and **USED BY** lines are generated from the real `import` statements in the code, so they're exact. **ALSO** lists links that aren't imports: HTTP calls, config files, things read from disk.

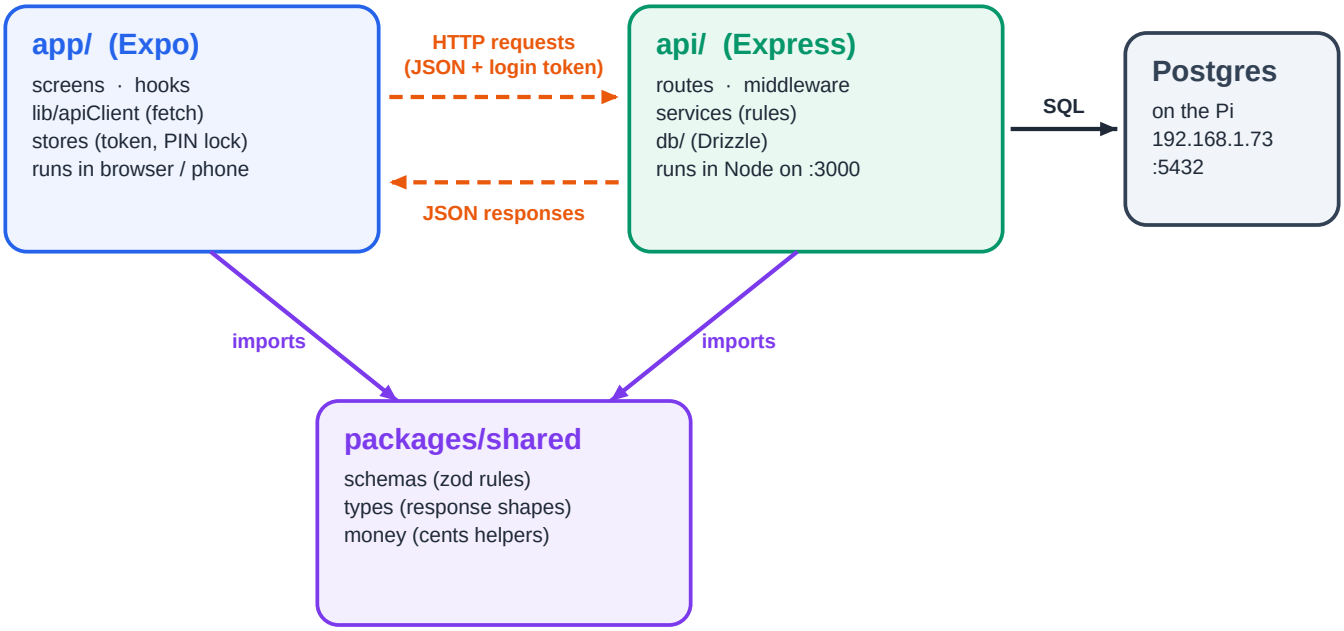
Blue	the Expo app (`app/`)
Purple	the shared package (`packages/shared/`)
Green	the server (`api/`)
Grey	repo-level config and tooling

Paths on the cards are shortened: `api/` means `api/src/`, `shared/` means `packages/shared/src/`, and `app/(tabs)/…` means `app/app/(tabs)/…`. **The last page** is the full map, a large landscape sheet with every file and every connection.

START HERE

The big picture

The project is **three npm workspaces in one repo**. The **app** is what you see. The **api** holds the rules and the only key to the database. **shared** is the common language they both import, so a deposit means the same thing on both sides.



Three rules that shape every file

- **Dependencies only point inward:** app → shared ← api. shared imports neither of them, and the app never imports API code: it only talks to it over HTTP. That's why Drizzle and pg never end up in the phone bundle, and why `types.ts` is written by hand.
- **Never trust the client:** the app checks input for a friendly experience, but the API checks it *again* (same zod schemas) and makes every money decision: caps, balance, family ownership.
- **Each layer has one job:** routes speak HTTP, services apply rules, db/ talks SQL. In the app, screens draw, hooks fetch, apiClient does the network, and stores remember. When you add something, ask "which layer is this?"

Two kinds of state in the app

Server data (balances, transactions, PIN status) lives in **React Query's cache**. It's a copy of what the API knows, refetched when marked stale. **App-only data** (the login token, "is the PIN unlocked?") lives in **zustand stores**. The token is saved to storage so you stay logged in; the PIN lock is not, so a restart re-locks it.

HOW THE FILES CONNECT AT RUNTIME

Follow a request

Imports show which file *can* use which. These two walkthroughs show what actually *happens*, file by file, when you use the app.

1. Logging in

1	`app/(auth)/login.tsx`	You tap **Log In**. The screen calls `mutate({ email, password })` from `useLoginMutation`.
2	`app/hooks/useAuthMutations.ts`	The mutation's `mutationFn` calls `postJson('/auth/login', input)`. `LoginInput` (from shared) makes sure the shape is right.
3	`app/lib/apiClient.ts`	`fetch` sends a POST to `http://localhost:3000/auth/login`.
4	`~ network ~`	The request travels to the API process.
5	`api/app.ts`	CORS allows the Expo origin, JSON is parsed, and the `/auth` prefix hands it to the auth router. (`index.ts` started all of this and ran migrations first.)
6	`api/routes/auth.routes.ts`	`LoginInputSchema.safeParse` (from shared) checks the body. If it's bad: **400** with a readable message.
7	`api/services/authService.ts`	Finds the parent with `db` (`db/client.ts` + `schema.ts`), runs `bcrypt.compare`, and signs a JWT with `JWT_SECRET` from `env.ts`.
8	`app/lib/apiClient.ts`	The **200** `{ token, parent }` comes back. `handleResponse` returns it (or throws the error message for 401).
9	`app/store/authStore.ts`	`onSuccess: saveSession` stores the token, which is saved to AsyncStorage.
10	`app/_layout.tsx`	`token` is now set, so `Stack.Protected` swaps `(auth)` for `(tabs)`. You're in.

2. Depositing $5 (the hooks you built today)

1	`app/(tabs)/deposit (Step 8)`	*(The deposit screen doesn't exist yet.)* It will call `useCreateTransactionMutation(childId).mutate({ type: 'deposit', amountCents: toCents(5), category: 'allowance' })`.
2	`app/hooks/useTransactions.ts`	`postJsonAuth` to `/children/:childId/transactions`. The body is a shared `TransactionInput`.
3	`app/lib/apiClient.ts`	Adds `Authorization: Bearer <token>`, reading the token from `authStore`.
4	`api/routes/children.routes.ts`	`router.use(requireAuth)` runs first...
5	`api/middleware/requireAuth.ts`	...verifies the JWT and sets `req.parentId` (typed by `express.d.ts`). A bad token stops here with **401**.
6	`api/routes/children.routes.ts`	`ChildIdSchema` checks the id (404) and `TransactionInputSchema` checks the body (400).
7	`api/services/transactionService.ts`	`getFamilyForParent`, then inside one DB transaction: lock the child row, `getBalance`, check the cap and balance (withdrawals only), insert, and return `{ transaction, balance }`.
8	`api/routes/children.routes.ts`	Replies **201**, or **422** "You don't have enough money for that."
9	`app/hooks/useTransactions.ts`	`onSuccess` invalidates `['transactions', childId]`, so `useTransactionsQuery` refetches and the dashboard shows the new balance.

REPO ROOT

The repository root

The top folder ties the three parts together. It doesn't hold app code itself: it holds the settings that make **app**, **api** and **packages/shared** behave like one project, and the files that keep every machine set up the same way.

`package.json`

WHAT IT DOES	Declares the three **npm workspaces** (`app`, `api`, `packages/shared`). One `npm install` here installs all three and links the shared package into `node_modules/@digital-bank/shared`. Also holds Prettier for the whole repo.
WHY WE NEED IT	Without workspaces, the app and the API couldn't import `@digital-bank/shared` without publishing it to npm first.
ALSO	Links: `app/package.json`, `api/package.json`, `packages/shared/package.json`

`package-lock.json`

WHAT IT DOES	The exact version of every installed package (about 12,000 lines). Written by npm, never by hand.
WHY WE NEED IT	Every machine (Windows, Linux, later the Pi) installs identical versions, so "works on my machine" stays true everywhere. *Note: this Linux machine's npm 9.2.0 rewrites it, so check* `git status` *after installs.*

`.gitignore`

WHAT IT DOES	Lists what git must never track: `node_modules/`, every `.env` file, Expo's build output, `api/dist/`, signing keys.
WHY WE NEED IT	Secrets (database password, JWT secret) must never reach GitHub, and generated folders are huge and can be rebuilt with `npm install`.

`.prettierrc`

WHAT IT DOES	The formatting rules: single quotes, semicolons, 2-space indents, 100-character lines.
WHY WE NEED IT	VS Code (Windows) and LazyVim (Linux) both read it, so code looks the same whichever editor saved it.
ALSO	Read by: `.vscode/settings.json` (format on save), LazyVim's prettier extra

`compose.dev.yml`

WHAT IT DOES	A Docker Compose file that runs Postgres 16 in a local container.
WHY WE NEED IT	This was the first dev database. Since 2026-09-30 the shared dev database lives on the Pi, so this is now an **offline fallback** (normally stopped).

`.vscode/settings.json`

WHAT IT DOES	VS Code settings: format with Prettier on every save.
WHY WE NEED IT	Editor setup for the Windows machine. LazyVim does the same thing on Linux.

`.claude/todo.md + .claude/session-logs/`

WHAT IT DOES	Your project diary: the running todo list and one log per working day.
WHY WE NEED IT	`session-start` reads them to pick up where you left off, and `session-end` writes them. Not code: nothing imports them.

SHARED

packages/shared: the common language

The small package that **both** the app and the API import. It holds the things both sides must agree on: what valid input looks like, what responses look like, and how money is handled. The golden rule: **shared imports nothing from app or api**. Arrows only point *into* it.

packages/shared/package.json

WHAT IT DOES	Names the package `@digital-bank/shared`. `main` and `types` point straight at `src/index.ts`. Depends on zod.
WHY WE NEED IT	Pointing at the TypeScript source means there's **no build step**: the API (via tsx) and the app (via Metro) read the .ts files directly.

packages/shared/tsconfig.json

WHAT IT DOES	TypeScript settings for checking the shared code by itself.
WHY WE NEED IT	Lets you run `npx tsc --noEmit -p packages/shared` to type-check just this package.

packages/shared/src/index.ts

WHAT IT DOES	The front door: re-exports everything from `types`, `schemas` and `money`.
WHY WE NEED IT	Other files write `from '@digital-bank/shared'` without needing to know which internal file something lives in.

IMPORTS	`shared/types.ts`
	`shared/schemas.ts`
	`shared/money.ts`

USED BY	`api/routes/auth.routes.ts`
	`api/routes/children.routes.ts`
	`api/routes/pin.routes.ts`
	`api/services/authService.ts`
	`api/services/transactionService.ts`
	`app/hooks/useAuthMutations.ts`
	`app/hooks/usePinMutations.ts`
	`app/hooks/useTransactions.ts`

packages/shared/src/schemas.ts

WHAT IT DOES	zod schemas, plus the TypeScript types inferred from them: `SignupInput` (exactly one of family name / invite code), `LoginInput`, `PinInput` (4 digits), `CreateChildInput`, `TransactionInput` (deposit needs a category; withdrawal needs a comment).
WHY WE NEED IT	One definition of "valid input" used on both sides. The API checks every request with `safeParse`, and the app's hooks use the same types, so the form and the server can never disagree about the rules.
USED BY	`shared/index.ts`

`packages/shared/src/types.ts`

WHAT IT DOES	The shapes of API **responses**: `Child` and `Transaction`. Dates are typed as `string`. *Written 2026-10-14 (Step 6).*
WHY WE NEED IT	So the app's hooks know exactly what JSON comes back. Hand-written instead of imported from the API's Drizzle schema, because shared must never depend on api. Dates are strings because JSON has no date type.
USED BY	`shared/index.ts`

`packages/shared/src/money.ts`

WHAT IT DOES	`toCents`, `fromCents`, `formatCents` (turns 1250 into "$12.50").
WHY WE NEED IT	All money is stored as **whole cents** (integers), because decimals in JavaScript round badly (0.1 + 0.2 = 0.30000000000000004). *Not used yet: the dashboard (Step 7) and the deposit screen (Step 8) will be the first users.*
USED BY	`shared/index.ts`

API

api/: the server

An Express server in Node. It's the only part that touches the database, and the only part that can be trusted, because a phone or browser can be tampered with. Requests flow through it in layers: **app.ts** picks a router by URL, **middleware** checks the login token, **routes** speak HTTP (status codes, request and response), **services** apply the business rules, and **db/** talks to Postgres.

Configuration and startup

api/package.json

WHAT IT DOES	The API's dependencies (express, drizzle-orm, pg, bcrypt, jsonwebtoken, cors, dotenv, zod) and scripts. `npm run dev` = `tsx watch`, which restarts the server on every save.
WHY WE NEED IT	tsx runs TypeScript directly, with no build step while developing. Remember it does **not** type-check: run `npx tsc --noEmit -p api` for that.

api/tsconfig.json

WHAT IT DOES	TypeScript settings for the server (strict mode, CommonJS modules).
WHY WE NEED IT	`tsc` uses it to check and build. *It has no* `lib` *setting, so browser (DOM) types leak in. That's the "add lib ES2020" todo item.*

api/.env (not in git)

WHAT IT DOES	Secrets and settings: `DATABASE_URL` (the Pi database), `JWT_SECRET`, `CORS_ORIGIN`.
WHY WE NEED IT	Keeps passwords out of the code and out of GitHub. Each machine has its own copy. Rebuilt from the Pi on 2026-10-14.
ALSO	Read by: `env.ts`, `db/client.ts`, `drizzle.config.ts` (via dotenv)

api/drizzle.config.ts

WHAT IT DOES	Settings for the **drizzle-kit command-line tool** (not the running server): where the schema is, where migrations go, which database to use.
WHY WE NEED IT	When you change `schema.ts`, `npx drizzle-kit generate` compares it with the last snapshot and writes a new SQL migration file.
ALSO	Reads: `src/db/schema.ts` Writes: `src/db/migrations/`

api/src/index.ts

WHAT IT DOES	The entry point. Runs any pending database migrations, *then* starts listening on port 3000.
WHY WE NEED IT	Migrating first means the tables always match the code before the first request arrives. It's kept separate from `app.ts` so the app can be built without opening a port (handy for tests later).
IMPORTS	`api/app.ts` `api/db/migrate.ts`

api/src/app.ts

WHAT IT DOES	Builds the Express app: CORS (only the Expo web address may call it), JSON body parsing, then mounts the routers at `/auth`, `/pin`, `/children`, plus `GET /health`.
WHY WE NEED IT	It's the switchboard: the first part of the URL decides which routes file handles the request.

IMPORTS	`api/env.ts`
	`api/routes/auth.routes.ts`
	`api/routes/pin.routes.ts`
	`api/routes/children.routes.ts`

USED BY	`api/index.ts`

api/src/env.ts

WHAT IT DOES	Loads `.env`, then exports `JWT_SECRET` (the server refuses to start without it) and `CORS_ORIGIN` (with a default).
WHY WE NEED IT	One place for configuration. It was pulled out of authService so that `requireAuth` uses the *same* secret check.

USED BY	`api/app.ts`
	`api/middleware/requireAuth.ts`
	`api/services/authService.ts`

Security: who is asking?

api/src/middleware/requireAuth.ts

WHAT IT DOES	The doorman. Reads the `Authorization: Bearer <token>` header, verifies the JWT with `JWT_SECRET`, puts the parent's id on `req.parentId`, then lets the request through. Otherwise it replies 401.
WHY WE NEED IT	Every protected route needs to know which parent is calling. Doing it once here means no route can forget to check.
IMPORTS	`api/env.ts`
USED BY	`api/routes/children.routes.ts` `api/routes/pin.routes.ts`

api/src/types/express.d.ts

WHAT IT DOES	Teaches TypeScript that Express requests can have a `parentId` property ("declaration merging").
WHY WE NEED IT	Without it, `req.parentId` would be a type error. It has no imports: TypeScript picks it up globally, which is why it shows as a dotted link on the map.
ALSO	Gives types to: `requireAuth.ts` and every route that reads `req.parentId`

Routes: the HTTP layer

api/src/routes/auth.routes.ts

WHAT IT DOES	`POST /auth/signup` and `POST /auth/login`. Each one validates the body (400 with a readable message), calls authService, and turns its error codes into status codes: 409 email taken, 501 invite signup, 401 bad login.
WHY WE NEED IT	This is the only router without `requireAuth`: you can't have a token before you've logged in.
IMPORTS	`api/services/authService.ts` `shared/index.ts`
USED BY	`api/app.ts`
ALSO	Called over HTTP by: `app/hooks/useAuthMutations.ts`

api/src/routes/pin.routes.ts

WHAT IT DOES	`GET /pin/status`, `POST /pin/set`, `POST /pin/verify`, each behind requireAuth. Maps PIN errors to 409 (not set or already set), 423 (locked) and 401 (wrong PIN).
WHY WE NEED IT	The PIN keeps kids out of the parent-only parts of the app.

IMPORTS	`api/services/pinService.ts`
	`shared/index.ts`
	`api/middleware/requireAuth.ts`

USED BY	`api/app.ts`
ALSO	Called over HTTP by: `app/hooks/usePinMutations.ts`

api/src/routes/children.routes.ts

WHAT IT DOES	`router.use(requireAuth)` protects everything in it. `GET/POST /children` and `GET/POST /children/:childId/transactions`. `ChildIdSchema` turns a malformed id into a clean 404. A refused withdrawal gets a 422 with a kid-friendly message.
WHY WE NEED IT	The money routes. It imports `db` directly only to pass it into `getBalance` and `listRecentTransactions` as the "executor" (the thing that runs the queries).

IMPORTS	`api/middleware/requireAuth.ts`
	`shared/index.ts`
	`api/services/childService.ts`
	`api/services/transactionService.ts`
	`api/db/client.ts`

USED BY	`api/app.ts`
ALSO	Called over HTTP by: `app/hooks/useTransactions.ts`

Services: the business rules

api/src/services/authService.ts

WHAT IT DOES	`signup`: checks the email isn't taken, hashes the password with bcrypt, and creates the family and parent together in one database transaction. `login`: checks the password with bcrypt. Both return a JWT that lasts 30 days.
WHY WE NEED IT	Passwords are never stored, only hashes. Unknown email and wrong password give the *same* error, so nobody can probe which emails have accounts.

IMPORTS	`shared/index.ts`
	`api/db/client.ts`
	`api/db/schema.ts`
	`api/env.ts`

USED BY	`api/routes/auth.routes.ts`

api/src/services/familyService.ts

WHAT IT DOES	`getFamilyForParent(parentId)`: looks up the parent, then their family.
WHY WE NEED IT	The family is the security boundary: every child or PIN action first asks "which family is yours?". It was pulled out of pinService so three services can share it.
IMPORTS	`api/db/client.ts` `api/db/schema.ts`

USED BY	`api/services/childService.ts`
	`api/services/pinService.ts`
	`api/services/transactionService.ts`

api/src/services/pinService.ts

WHAT IT DOES	PIN status, set (once only) and verify. Five wrong tries lock it for 5 minutes (`pinAttempts` / `pinLockedUntil` on families).
WHY WE NEED IT	The PIN belongs to the family, not to each parent, so a co-parent uses the same one. The lockout stops a kid from guessing all 10,000 combinations.

IMPORTS	`api/db/client.ts`
	`api/db/schema.ts`
	`api/services/familyService.ts`

USED BY	`api/routes/pin.routes.ts`

api/src/services/childService.ts

WHAT IT DOES	`createChild`, `listChildren` (not archived, oldest first) and `getChildForParent` (the ownership check: throws `CHILD_NOT_FOUND` for anyone else's child).
WHY WE NEED IT	Every query is filtered by family id, so one family can never see or touch another family's kids.

IMPORTS	`api/db/client.ts`
	`api/db/schema.ts`
	`api/services/familyService.ts`

USED BY	`api/routes/children.routes.ts`

api/src/services/transactionService.ts

WHAT IT DOES	`getBalance`: deposits minus withdrawals (the balance is **calculated, never stored**). `listRecentTransactions`: the newest 20. `createTransaction`: locks the child's row, checks the spending cap, then the balance, then inserts.
WHY WE NEED IT	The row lock (`FOR UPDATE`) makes simultaneous withdrawals wait their turn, which you tested: 5 at once on $10 gave exactly 3 successes. Calculating the balance means it can never drift out of step with the transaction history.

IMPORTS	`api/db/client.ts`
	`api/db/schema.ts`
	`api/services/familyService.ts`
	`shared/index.ts`

USED BY	`api/routes/children.routes.ts`

Database

api/src/db/schema.ts

WHAT IT DOES	The four tables written in TypeScript: `families`, `parents`, `children`, `transactions`. Also their links (family_id foreign keys), CHECK rules (amount > 0; type is deposit or withdrawal) and an index for "newest transactions per child".
WHY WE NEED IT	The single source of truth for the database. Drizzle uses it to type every query, and drizzle-kit uses it to write migrations.

USED BY	`api/db/client.ts`
	`api/services/authService.ts`
	`api/services/childService.ts`
	`api/services/familyService.ts`
	`api/services/pinService.ts`
	`api/services/transactionService.ts`

api/src/db/client.ts

WHAT IT DOES	Opens a pool of Postgres connections from `DATABASE_URL` and creates the Drizzle `db` object.
WHY WE NEED IT	One shared pool for the whole server. Every service imports `db` from here instead of opening its own connection.
IMPORTS	`api/db/schema.ts`

USED BY	`api/db/migrate.ts`
	`api/routes/children.routes.ts`
	`api/services/authService.ts`
	`api/services/childService.ts`
	`api/services/familyService.ts`
	`api/services/pinService.ts`
	`api/services/transactionService.ts`

ALSO	Connects to: Postgres on the Pi (192.168.1.73:5432)

api/src/db/migrate.ts

WHAT IT DOES	Applies any migration files the database hasn't seen yet. Drizzle records which ones ran in a table of its own.
WHY WE NEED IT	Called by `index.ts` on every start, so a new migration is applied automatically. *Its folder path is relative, so start the API from* `api/` *(which* `npm run dev` *does).*
IMPORTS	`api/db/client.ts`
USED BY	`api/index.ts`
ALSO	Reads: `src/db/migrations/`

`api/src/db/migrations/ (2 .sql files + meta/)`

WHAT IT DOES	`0000_soft_the_anarchist.sql` creates families and parents (Phase 1). `0001_premium_landau.sql` creates children and transactions (Phase 2). `meta/` holds drizzle-kit's bookkeeping: the order of migrations and a snapshot of the schema after each one.
WHY WE NEED IT	Migrations are the step-by-step history of the database, so any empty database (a new machine, the Pi in Phase 4) can be built up to today's shape. Generated files with random names: never edit them by hand. *tsx watch doesn't notice new ones, so restart the API.*

APP

app/: the Expo app

The React Native app, built with Expo. It runs on the web now and on Android later. It holds no secrets and makes no money decisions: it shows data and sends requests. It uses two kinds of state: **server data** (balances, PIN status) lives in React Query's cache, and **app-only state** (your login token, whether the PIN is unlocked) lives in small zustand stores.

Configuration

app/package.json

WHAT IT DOES	Expo SDK 54, React Native 0.81, expo-router, React Query, zustand, AsyncStorage (pinned at 2.2.0 for SDK 54). `"main": "expo-router/entry"` means Expo Router starts the app from the `app/app/` folder.
WHY WE NEED IT	The scripts pin port 19006, because 5173 belongs to another project.

app/app.json

WHAT IT DOES	Expo's app settings: the name, the `digitalbank://` link scheme, the Android package id, the Metro bundler for web, the expo-router plugin.
WHY WE NEED IT	Expo reads this to know how to build and run the app on each platform.

app/tsconfig.json

WHAT IT DOES	Builds on Expo's TypeScript settings and defines the `@/` shortcut, which points at the `app/` folder.
WHY WE NEED IT	`@/lib/apiClient` reads better than `../../lib/apiClient` and doesn't break when files move.

app/babel.config.js

WHAT IT DOES	Expo's Babel preset with `unstable_transformImportMeta` switched on.
WHY WE NEED IT	This fixes the blank web screen: zustand uses `import.meta`, which Metro's web bundle couldn't read until this setting was added.

Screens: app/app/ (Expo Router)

`app/app/_layout.tsx`

WHAT IT DOES	The root of every screen. It wraps the app in `QueryClientProvider`, shows nothing until the saved login has loaded, then uses `Stack.Protected` to decide: token means `(tabs)`, no token means `(auth)`. Also starts `useAutoLock`.
WHY WE NEED IT	This is the login gate for the whole app. **File-based routing:** files under `app/app/` *are* the screens. Folders in (parentheses) group screens without changing the URL, and a `_layout.tsx` wraps the screens in its folder. *Route files must live under app/app/, not app/ (the September stray-folder gotcha).*

IMPORTS	`app/lib/queryClient.ts`
	`app/store/authStore.ts`
	`app/hooks/useAutoLock.ts`

`app/app/(auth)/_layout.tsx`

WHAT IT DOES	A plain screen stack, with no headers, for login and signup.
WHY WE NEED IT	Groups the logged-out screens so the root layout can show or hide them together.

`app/app/(auth)/login.tsx`

WHAT IT DOES	The email and password form. It calls `useLoginMutation`, shows "Logging in..." while waiting and the error message if login fails, and links to signup.
WHY WE NEED IT	How a parent gets a token.
IMPORTS	`app/hooks/useAuthMutations.ts`

`app/app/(auth)/signup.tsx`

WHAT IT DOES	A "Create a family" / "Join with invite code" toggle, plus email and password.
WHY WE NEED IT	Creates the family and the first parent. The invite branch gets a 501 until Phase 3.
IMPORTS	`app/hooks/useAuthMutations.ts`

`app/app/(tabs)/_layout.tsx`

WHAT IT DOES	The bottom tab bar: Home and Settings, with icons.
WHY WE NEED IT	The main navigation once logged in.

`app/app/(tabs)/index.tsx`

WHAT IT DOES	The Home tab. Currently a placeholder.
WHY WE NEED IT	**Step 7 builds the dashboard here**, using `useChildrenQuery`, `useTransactionsQuery` and `formatCents`.

app/app/(tabs)/settings.tsx

WHAT IT DOES	A PIN gate with three states: *set a PIN*, then *enter the PIN*, then the real settings. After a PIN is set it refreshes `['pinStatus']`, and it clears the box after every attempt.
WHY WE NEED IT	Keeps parent settings (caps, children, invites) away from kids.
IMPORTS	`app/hooks/usePinMutations.ts` `app/store/pinStore.ts`

Hooks: how screens talk to the server

app/hooks/useAuthMutations.ts

WHAT IT DOES	`useSignupMutation` and `useLoginMutation`. When either succeeds, it saves the token and parent into authStore.
WHY WE NEED IT	Screens just call `mutate(...)`. The hook handles the request and saving the session.

IMPORTS	`shared/index.ts`
	`app/lib/apiClient.ts`
	`app/store/authStore.ts`

USED BY	`app/(auth)/login.tsx` `app/(auth)/signup.tsx`
ALSO	HTTP to: `api/src/routes/auth.routes.ts`

app/hooks/usePinMutations.ts

WHAT IT DOES	`usePinStatusQuery` (a read, so `useQuery`) and `useSetPinMutation` / `useVerifyPinMutation`.
WHY WE NEED IT	Same pattern for the PIN routes. *Reads are queries, changes are mutations.*
IMPORTS	`shared/index.ts` `app/lib/apiClient.ts`
USED BY	`app/(tabs)/settings.tsx`
ALSO	HTTP to: `api/src/routes/pin.routes.ts`

app/hooks/useTransactions.ts

WHAT IT DOES	`useChildrenQuery`, `useTransactionsQuery(childId)` (waits until a child is known) and `useCreateTransactionMutation(childId)`, which marks that child's data stale after success so the balance refreshes. *Written 2026-10-14 (Step 6).*
WHY WE NEED IT	The bridge between the money screens (Steps 7 and 8) and the money routes. The cache key `['transactions', childId]` is what links the mutation to the query.
IMPORTS	`shared/index.ts` `app/lib/apiClient.ts`
ALSO	HTTP to: `api/src/routes/children.routes.ts`

app/hooks/useAutoLock.ts

WHAT IT DOES	Watches `AppState`. As soon as the app stops being active (switched away, browser tab hidden), it locks the PIN.
WHY WE NEED IT	A parent who unlocks Settings and hands the phone to a kid isn't leaving it unlocked.
IMPORTS	`app/store/pinStore.ts`
USED BY	`app/_layout.tsx`

lib/ and store/: plumbing and memory

app/lib/apiClient.ts

WHAT IT DOES	The **only** file that calls `fetch`. It knows the API's address (`EXPO_PUBLIC_API_URL` or localhost:3000). `handleResponse` turns error replies into `Error(message)`. `postJson` sends no token; `getJsonAuth` and `postJsonAuth` attach the token from authStore.
WHY WE NEED IT	One place for the address, headers and error handling. That error message is what every form shows, which is why readable 400 messages matter.
IMPORTS	`app/store/authStore.ts`

USED BY	`app/hooks/useAuthMutations.ts`
	`app/hooks/usePinMutations.ts`
	`app/hooks/useTransactions.ts`

ALSO	HTTP to: the API on port 3000

app/lib/queryClient.ts

WHAT IT DOES	Creates the one React Query cache for the whole app.
WHY WE NEED IT	`_layout.tsx` provides it to every screen. Invalidating a key affects every screen using that key.
USED BY	`app/_layout.tsx`

app/store/authStore.ts

WHAT IT DOES	A zustand store holding the token and parent, saved to AsyncStorage under `auth-storage`. `hasHydrated` becomes true once the saved copy has loaded.
WHY WE NEED IT	You stay logged in across reloads. AsyncStorage was chosen because it also works on the web, where expo-secure-store doesn't.

USED BY	`app/_layout.tsx`
	`app/hooks/useAuthMutations.ts`
	`app/lib/apiClient.ts`

`app/store/pinStore.ts`

WHAT IT DOES	One `unlocked` true/false value with `unlock()` and `lock()`. Deliberately **not** saved.
WHY WE NEED IT	Every restart starts locked, so the PIN unlock only lasts one session.
USED BY	`app/(tabs)/settings.tsx` `app/hooks/useAutoLock.ts`

ODDS AND ENDS

Not in git, and things I noticed

Folders and files that exist on disk but are not committed

`node_modules/` Every installed package, rebuilt by `npm install`. `node_modules/@digital-bank/shared` is a *link* back to `packages/shared`, which is how the imports resolve.

`api/.env` The API's secrets (see the API section).

`app/.expo/` Expo's cache and generated route types. Safe to delete; Expo recreates it.

Things I noticed while mapping

None of these are urgent, and nothing is broken today. They're worth knowing about.

`api/src/services/transactionService.ts:6`	`import { error } from 'console';` is never used. It's probably an accidental editor auto-import, and it's safe to delete.
`api/src/services/familyService.ts`	It assumes the parent exists. A valid token for a parent who has since been deleted would crash with a TypeError, giving a generic 500 instead of a 401. Worth handling when "remove co-parent" arrives in Phase 3.
`packages/shared/src/money.ts`	Nothing imports it yet. It has no arrows on the map, and the dashboard in Step 7 will be its first user.
`app/app/(tabs)/_layout.tsx, index.tsx, db/client.ts, shared index.ts, money.ts`	These 5 files predate Prettier (they use double quotes). They're already on the todo list as a formatting-only commit.
`app/package.json vs api/package.json`	The app asks for shared as `"^0.0.0"` and the API as `"*"`. Both work with workspaces, so it's cosmetic only.

Next page: the full map. It's printed on a larger 11×17 landscape sheet so every label stays readable.

Digital Bank: full project map

Every source file and every import between them (generated from the code), plus HTTP calls and non-import links. Arrows point from the file that uses to the file being used. 2026-10-14

imports (colour = workspace of the importing file)
HTTP request over the network
non-import link (types, config, files, SQL)
planned (not written yet)

app/ Expo app: screens, hooks, plumbing

packages/shared

api/ Express server: entry, routes, services, database

POST /auth/signup · /auth/login
GET /children · GET + POST /children/:id/transactions
GET /pin/status · POST /pin/set · /pin/verify

SCREENS (APP/APP/)

_layout.tsx
app/app/ root layout

_layout.tsx
app/app/(auth)/

login.tsx
app/app/(auth)/

signup.tsx
app/app/(auth)/

_layout.tsx
app/app/(tabs)/ tab bar

index.tsx
app/app/(tabs)/ Home
tap 7

settings.tsx
app/app/(tabs)/ PIN gate

HOOKS

useAutoLock.ts
app/hooks/

useAuthMutations.ts
app/hooks/

useTransactions.ts
app/hooks/ Step 6

usePinMutations.ts
app/hooks/

LIB + STORE

pinStore.ts
app/store/ not saved

queryClient.ts
app/lib/ RQ cache

apiClient.ts
app/lib/ only fetch()

authStore.ts
app/store/ saved token

SHARED

index.ts
shared/src/ front door

schemas.ts
shared/src/ zod rules

types.ts
shared/src/ Step 6

money.ts
shared/src/ not used yet

ENTRY + CONFIG

index.ts
api/src/ entry point

app.ts
api/src/ switchboard

env.ts
api/src/ reads .env

express.d.ts
api/src/types/

ROUTES + MIDDLEWARE

auth.routes.ts
api/src/routes/ /auth

pin.routes.ts
api/src/routes/ /pin

children.routes.ts
api/src/routes/ /children

requireAuth.ts
api/src/middleware/

types req.parentId

SERVICES

authService.ts
api/src/services/

pinService.ts
api/src/services/

childService.ts
api/src/services/

transactionService.ts
api/src/services/

familyService.ts
api/src/services/

DATABASE

migrate.ts
api/src/db/

client.ts
api/src/db/ the db pool

schema.ts
api/src/db/ 4 tables

reads

migrations/
api/src/db/ 0000, 0001, meta

applies writes

drizzle.config.ts
api/ drizzle-kit CLI

Postgres
on the Pi 192.168.1.73

SQL

Config and tooling

Not imported by any code file. Read by npm, Expo, TypeScript, Prettier or Docker instead.

package.json
workspaces: links app, api, shared

package-lock.json
exact versions of every package

.gitignore
keeps .env + node_modules out of git

.prettierrc
format rules: VS Code + LazyVim

compose.dev.yml
local Postgres (offline fallback)

.vscode/settings.json
format on save (Windows)

shared/package.json
main: src/index.ts (no build step)

shared/tsconfig.json
tsc -p packages/shared

app/package.json
main: expo-router/entry · port 19006

app/app.json
Expo config: name, scheme, android

app/tsconfig.json
defines the @/ alias for app imports

app/babel.config.js
import.meta fix (zustand on web)

api/package.json
npm run dev = tsx watch src/index.ts

api/tsconfig.json
strict TS · (todo: lib ES2020)

api/.env (not in git)
read by env.ts, client.ts, drizzle.config

.claude/todo + logs
session-start reads · session-end writes